GitHub Beyond Add & Push

A working developer's guide to Git & GitHub for full-stack roles

Prepared for Mohanraj R

Personal skill reference — not a classroom handout

How to use this guide

This isn't the add/commit/push story again. It's what changes once you're on a team: branching discipline, pull requests, code review, cleaning up messy history, undoing mistakes without panic, and the CI/CD layer that real repos run on. Each section is written the way you'll actually hit the topic on the job — as a command sequence and a decision to make, not trivia.

Read it top to bottom once, then keep it as a reference. The command tables at the end of most sections are what you'll come back to.

1. The mental model you actually need

You know add/commit/push mechanically. The thing that unlocks everything else — rebase, cherry-pick, reflog, resetting — is this one idea:

> **Core idea**
> A commit is a snapshot, not a diff. A branch is just a movable pointer (a label) to one commit. HEAD is a pointer to "the branch you're currently on." Almost every advanced Git command is really just moving pointers around or copying snapshots — nothing mystical.

Three areas matter for every operation:

- Working directory — the files you see and edit
- Staging area / index — what git add puts there; the draft of your next commit
- Repository (.git) — the committed history, immutable snapshots

Once you see rebase as "replay these commits on top of a different base" and reset as "move the branch pointer, and optionally the staging area and files too," the rest of this guide is just vocabulary.

2. Branching strategy in real teams

No serious team commits to main directly. You'll be expected to know which flavor of branching a repo uses within your first week.

GitHub Flow (most common in product/startup teams)

- main is always deployable
- Every piece of work gets a short-lived branch off main
- Open a PR early, merge after review, delete the branch
- Deploy straight from main, often automatically

Git Flow (larger, release-based products)

- Long-lived develop branch alongside main
- feature/* branches off develop, release/* branches to stabilize, hotfix/* off main for emergencies
- More ceremony — you'll meet this in enterprise or versioned-release products

Trunk-based development

- Everyone commits small, frequent changes straight to main (or very short branches, <1 day)
- Relies heavily on feature flags to hide unfinished work in production
- Common at companies optimizing for continuous deployment

Branch naming — the convention you'll be expected to follow

```
feature/user-authentication
fix/cart-total-rounding-bug
chore/upgrade-react-18
hotfix/prod-login-500
refactor/extract-payment-service
```

Prefix by intent (feature/fix/chore/hotfix/refactor), then a short kebab-case description. Some teams prepend a ticket number: feature/JIRA-482-user-auth.

3. Commit hygiene

In a team repo, your commit history is documentation other engineers read during debugging and code review. Sloppy history costs everyone time.

Atomic commits

One logical change per commit. "Add login form" and "fix typo in login form" should usually be one commit, not two, by the time you open a PR — see interactive rebase below.

Conventional Commits (widely used, plays well with automated changelogs)

```
feat: add JWT refresh token endpoint
fix: correct off-by-one in pagination
refactor: extract validation into middleware
chore: bump axios to 1.7.2
docs: update API README
test: add unit tests for cart service
BREAKING CHANGE: renamed /api/user to /api/users
```

Amending your last commit

```
git add forgotten-file.js
git commit --amend --no-edit
```

Only amend commits that haven't been pushed and shared yet — amending rewrites history.

Interactive rebase — squash a messy branch before opening a PR

This is the single most useful "advanced" Git skill for a working developer. Say you have 6 messy WIP commits on your feature branch:

```
git rebase -i HEAD~6

# In the editor that opens:
pick a1b2c3 add login form skeleton
squash d4e5f6 wip
squash a7b8c9 fix typo
squash 1a2b3c actually fix typo
```

```
reword 9f8e7d add login form
squash 5c4b3a add tests
```

- pick — keep commit as is
- squash (s) — merge into the commit above, combine messages
- fixup (f) — merge into the commit above, discard its message
- reword (r) — keep the change, edit the message
- drop (d) — delete the commit entirely

Result: one (or a few) clean commits with a real message, instead of six "wip" commits polluting main's history forever.

4. Rebase vs merge — knowing which to reach for

> **Rule of thumb**
>
> Rebase your own local/feature branch to keep it clean and up to date. Merge when integrating into a shared branch (like main) that others already pulled from. Never rebase a branch other people are also working on.

Keeping your feature branch current with main

```
git checkout feature/user-auth
git fetch origin
git rebase origin/main

# if conflicts appear:
#  1. fix the conflicted files
git add <resolved-file>
git rebase --continue

# if it's going badly and you want out:
git rebase --abort
```

This replays your commits on top of the latest main, so your PR shows a clean, linear diff instead of a tangle of merge commits.

git pull --rebase instead of a plain pull

```
git pull --rebase origin main
```

Plain git pull does fetch + merge, which creates a merge commit every time you sync. Most teams prefer pull --rebase (or set it as default) to keep history linear:

```
git config --global pull.rebase true
```

Merge — when you actually want it

```
git checkout main
git merge feature/user-auth
```

This is effectively what clicking "Merge pull request" on GitHub does. It's the right tool once code is reviewed and landing into a branch other people build on.

5. Pull requests & code review — where the job actually happens

What makes a PR reviewable

- Small and focused — one feature or fix, not three unrelated changes
- A description that says what changed and why, plus how to test it
- Linked to the ticket/issue it resolves
- Passing CI before you request review (don't waste a reviewer's time on a red pipeline)

Draft PRs

Open a PR as a draft the moment you start meaningful work — it gives visibility, runs CI early, and invites early feedback without asking for a full review yet. Mark it "Ready for review" when it's actually ready.

Responding to review comments without messing up history

Don't just push new "fix review comments" commits forever. Two clean patterns:

```
# Pattern 1: fixup commits, then squash before merge
git commit --fixup <commit-sha>
git rebase -i --autosquash origin/main

# Pattern 2: amend if it's the most recent commit
git add .
git commit --amend --no-edit
git push --force-with-lease
```

Never use plain --force

Use git push --force-with-lease, never git push --force, on a shared branch. --force-with-lease refuses to overwrite work if someone else pushed to that branch since you last fetched. Plain --force can silently delete a teammate's commits.

Merge strategies on GitHub — pick correctly

Command	What it does
`Create a merge commit`	Preserves every commit + a merge commit. Good for release branches where full history matters.
`Squash and merge`	Collapses the whole PR into one commit on main. Most common default — keeps main's history readable, one PR = one commit.
`Rebase and merge`	Replays each PR commit individually onto main, no merge commit. Use when you want linear history and your commits are already clean.

CODEOWNERS — who gets auto-requested for review

```
# .github/CODEOWNERS
/frontend/          @mo-frontend-team
/backend/api/        @mo @backend-lead
*.sql                @db-team
```

Add this file to a repo and GitHub automatically requests reviews from the right people based on which files a PR touches.

6. Undoing mistakes without panic

This is the toolkit that separates someone comfortable with Git from someone who's afraid of it. Almost nothing in Git is actually destroyed immediately — reflog is your safety net.

git reset — three modes, know the difference

Command	What it does
`git reset --soft HEAD~1`	Undo the last commit, keep changes staged. Good for "I want to redo that commit message/content."
`git reset --mixed HEAD~1`	Undo the commit, unstage changes, keep them in working directory. Default mode.
`git reset --hard HEAD~1`	Undo the commit and discard the changes entirely. Destructive — only on commits not yet pushed/shared.

git revert — the safe undo for shared history

```
git revert <commit-sha>
```

Creates a new commit that undoes a previous one, without rewriting history. This is what you use to undo something already merged into main — reset would rewrite shared history and break everyone else's clone.

git reflog — your undo button for almost anything

```
git reflog
# shows every place HEAD has pointed recently, even after a hard reset
git reset --hard HEAD@{2}   # jump back to a state from a few actions ago
```

If you ever "lose" commits from a bad reset or rebase, reflog almost always still has them for a while. This single command has saved more careers than any other in Git.

git stash — parking work-in-progress

```
git stash push -m "wip: cart discount logic"
git checkout main       # go fix something urgent
git checkout feature/cart
git stash list
git stash pop            # restore + remove from stash
git stash apply stash@{1}  # restore but keep it in the stash list too
```

git cherry-pick — copy a specific commit onto another branch

```
git checkout hotfix/prod-issue
git cherry-pick a1b2c3d
```

Common real scenario: a fix landed on a feature branch but needs to go straight to a hotfix or release branch without pulling in everything else from that feature branch.

7. Collaborating without chaos

fetch vs pull

git fetch downloads new commits but doesn't touch your working branch — safe to run anytime to see what's new. git pull = fetch + merge (or rebase) immediately. Get in the habit of fetching first, looking at what changed, then deciding.

Keeping a fork in sync with upstream (open source / multi-repo work)

```
git remote add upstream https://github.com/original-org/repo.git
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

Resolving a real merge conflict

```
git status                      # shows conflicted files
# open the file, look for:
<<<<<<< HEAD
your version
=======
their version
>>>>>>> branch-name
# edit to the correct final content, remove the markers
git add <file>
git commit                      # (or rebase --continue if mid-rebase)
```

For anything beyond a one-line conflict, use a visual tool rather than fighting markers by eye:

```
git mergetool
# or open in VS Code — it shows "Accept Current / Incoming / Both" inline
```

8. Tags, releases & versioning

Semantic Versioning (semver) — what full-stack teams expect you to know

Given MAJOR.MINOR.PATCH, e.g. 2.4.1:

- MAJOR — breaking change, incompatible API
- MINOR — new backward-compatible feature
- PATCH — backward-compatible bug fix

Tagging a release

```
git tag -a v2.4.1 -m "Release 2.4.1 - fix cart rounding"
git push origin v2.4.1
git push origin --tags     # push all tags at once
```

On GitHub, go to Releases → Draft a new release, pick the tag, and GitHub can auto-generate release notes from merged PRs since the last tag.

9. GitHub Actions — the CI/CD layer you'll be expected to read and edit

Full-stack roles almost always involve touching a workflow file at some point — adding a test step, fixing a broken pipeline, adding a deploy step. You don't need to be a DevOps engineer, but you need to read and modify this comfortably.

Anatomy of a workflow

```
# .github/workflows/ci.yml
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
```

- on: — what triggers the workflow (PR opened, push to main, a schedule, manually via workflow_dispatch)
- jobs: — one or more parallel/sequential units of work, each running on a fresh VM
- steps: — sequential commands or reusable actions (uses:) inside a job
- actions/checkout@v4 is almost always your first step — it pulls the repo code onto the runner

A simple deploy job (common real pattern)

```
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - name: Deploy
        env:
          DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
        run: ./deploy.sh
```

needs: test means this job only runs after the test job passes. Secrets never live in the YAML — they're stored in repo Settings → Secrets and referenced as \${{ secrets.NAME }}.

Status checks gate merges

Once CI is set up, you'll typically see PRs blocked from merging until checks pass — that's Actions talking to branch protection rules (next section).

10. Branch protection & repo governance

Repo Settings → Branches → Add rule, usually applied to main:

- Require a pull request before merging (no direct pushes)
- Require approvals — e.g. at least 1–2 reviewers
- Require status checks to pass (your CI workflow) before merging
- Require branches to be up to date before merging
- Require signed commits (see below)
- Restrict who can push, and disallow force-pushes to the branch

.gitignore — get this right per stack

```
# Node/React
node_modules/
.env
.env.local
dist/
build/

# Python
__pycache__/
*.pyc
.venv/
venv/

# General
.DS_Store
*.log
```

Secrets management — the rule that matters most

> **Never commit secrets**
>
> API keys, DB passwords, and tokens go in .env (gitignored) locally, and in GitHub Actions Secrets / a secrets manager in CI and production — never hardcoded, never committed. If a secret does get committed, rotating/revoking it is mandatory — removing it from a later commit is not enough, since it still exists in history.

```
# if a secret was committed and needs to be purged from history:
git filter-repo --path secrets.env --invert-paths
# then force-push, and rotate the leaked credential regardless
```

Commit signing (SSH or GPG)

```
git config --global commit.gpgsign true
git config --global user.signingkey <your-key-id>
git commit -S -m "signed commit"
```

Some companies require signed commits so GitHub shows a "Verified" badge, proving the commit really came from your account and wasn't spoofed.

11. Issues & project boards

Linking commits and PRs to issues

```
git commit -m "fix: correct cart rounding error

Closes #42"
```

Using Closes #42, Fixes #42, or Resolves #42 in a PR description auto-closes that issue when the PR merges — this is expected practice, not optional flourish.

GitHub Projects (Kanban-style boards)

- Backlog → In Progress → In Review → Done columns tied directly to issues/PRs
- Automations move cards automatically (e.g. PR merged → issue moves to Done)
- Used for sprint planning in most agile teams — you'll be expected to move your own cards

12. Advanced topics you'll meet eventually

Submodules vs monorepo

Submodules embed one Git repo inside another at a pinned commit (common for shared libraries):

```
git submodule add https://github.com/org/shared-ui.git libs/shared-ui
git submodule update --init --recursive
```

Many full-stack teams instead use a monorepo (frontend + backend + shared packages in one repo, managed with tools like Nx or Turborepo) to avoid submodule pain — expect to encounter either.

Git hooks — automated checks before commit/push

```
# .husky/pre-commit  (using the Husky npm package)
npx lint-staged
```

Pre-commit hooks commonly run linting/formatting/tests automatically so bad code never even gets committed. If you clone a repo and commits keep getting blocked locally with an error, this is almost always why.

GitHub Codespaces

A full dev environment (VS Code + terminal + your repo) running in the cloud, launched straight from a repo's "Code" button — useful when onboarding to a new codebase without a local setup.

13. A realistic day: ticket to merged PR

```
# 1. Sync main
git checkout main
git pull --rebase origin main
```

```
# 2. Branch for the ticket
git checkout -b feature/JIRA-217-cart-discount

# 3. Work, committing in small logical chunks
git add src/cart/discount.js
git commit -m "feat: add percentage discount calculation"

# 4. Keep branch current while you work (daily, or before opening PR)
git fetch origin
git rebase origin/main

# 5. Clean up history before opening the PR
git rebase -i origin/main   # squash wip commits

# 6. Push and open the PR
git push -u origin feature/JIRA-217-cart-discount
# open PR on GitHub, link ticket, wait for CI

# 7. Respond to review feedback
git commit --fixup <sha>
git rebase -i --autosquash origin/main
git push --force-with-lease

# 8. Merge (squash and merge via GitHub UI), delete branch
git checkout main
git pull --rebase origin main
git branch -d feature/JIRA-217-cart-discount
```

14. Quick command reference

Command	What it does
`git fetch`	Download new commits without touching your branch
`git pull --rebase`	Sync your branch with remote, replaying local commits on top
`git rebase -i HEAD~n`	Interactively edit/squash/reorder the last n commits
`git rebase origin/main`	Replay your branch's commits on top of latest main
`git commit --amend`	Edit the most recent commit
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git reset --hard HEAD~1`	Undo last commit, discard changes (unpushed only)
`git revert <sha>`	Safely undo a commit already shared with others
`git reflog`	See history of where HEAD has been — recovery net
`git stash / stash pop`	Park and restore uncommitted work
`git cherry-pick <sha>`	Copy one commit onto the current branch
`git push --force-with-lease`	Force-push safely, refuses if remote has new commits
`git tag -a vX.Y.Z -m "..."`	Create an annotated release tag
`git mergetool`	Open a visual tool to resolve conflicts
`git log --oneline --graph --all`	Visualize branch/commit structure in the terminal

Next step: pick one real repo (yours or an open-source one) and deliberately practice interactive rebase, a rebase conflict, and a full PR-with-CI cycle. Reading this once won't make it muscle memory — running these commands with something actually at stake will.